Advanced Data Structures

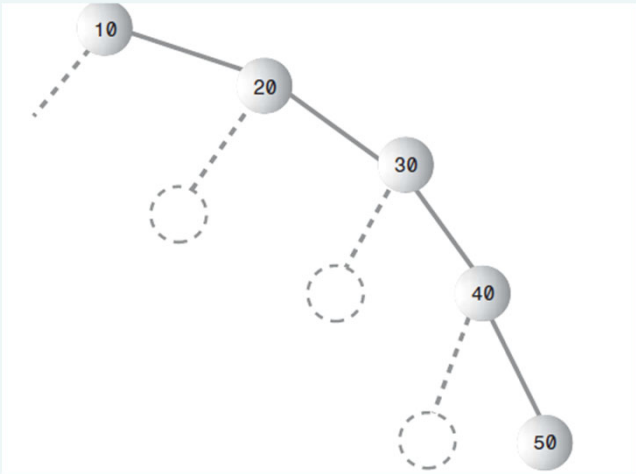
Balanced Search Trees

- Red-black trees
- Height of a red-black tree
- Rotations
- Insertion

Balanced and Unbalanced Trees

Binary Trees are balanced when the left and the right subtrees are almost at the same level.

Trees become unbalanced when you start inserting nodes either in ascending and descending order.



Balanced search trees

Balanced search tree: A search-tree data structure for which a height of $O(\lg n)$ is guaranteed when implementing a dynamic set of n items.

- Red-Black trees
- AVL trees
- 2-3 trees
- 2-4 trees
- B-trees

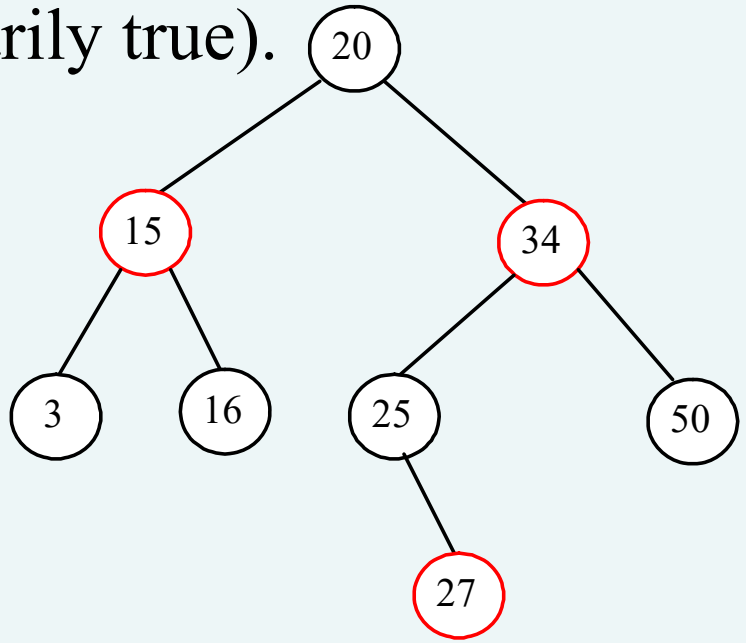
Red black trees

Characteristics:

- The nodes are colored
- During insertion and deletion, rules are followed that preserve the various arrangements of these colors.
- Ensures that the tree is always balanced thus preserving the efficiency.

Red-Black rules

1. Every node is either red or black
2. The root is always black.
3. If a node is red, its children must be black (although the converse isn't necessarily true).
4. Every path from the root to the leaf, or an external node, must contain the same number of black nodes. (black depth)



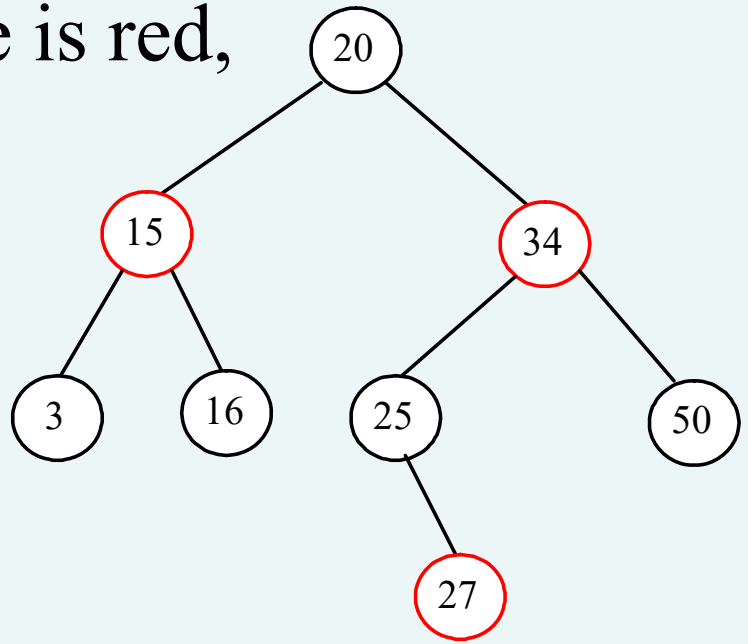
Modifying operations

The operations INSERT and DELETE cause modifications to the red-black tree:

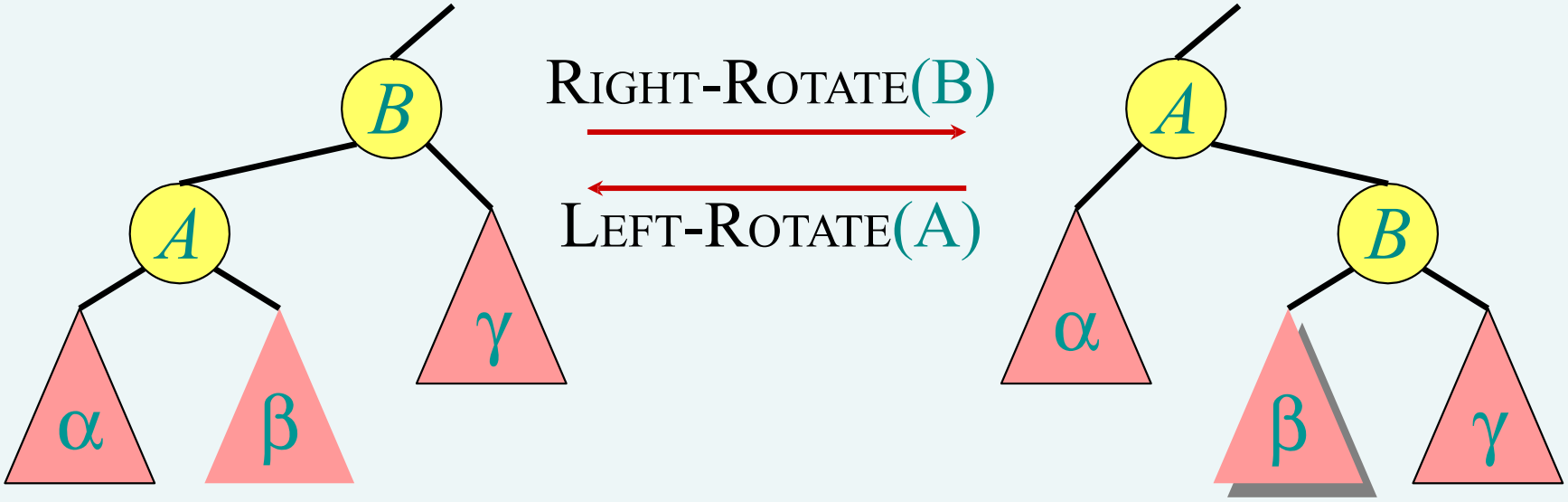
- the operation itself,
- color changes,
- restructuring the links of the tree via ***"rotations"***.

How to insert an element to red-black tree.

- A new node is always inserted as a leaf node.
  - If root, color it black, otherwise, color it red.
  - If the parent of the new node is red, it violates property 2 of the red-black tree. (double-red violation)



Rotations



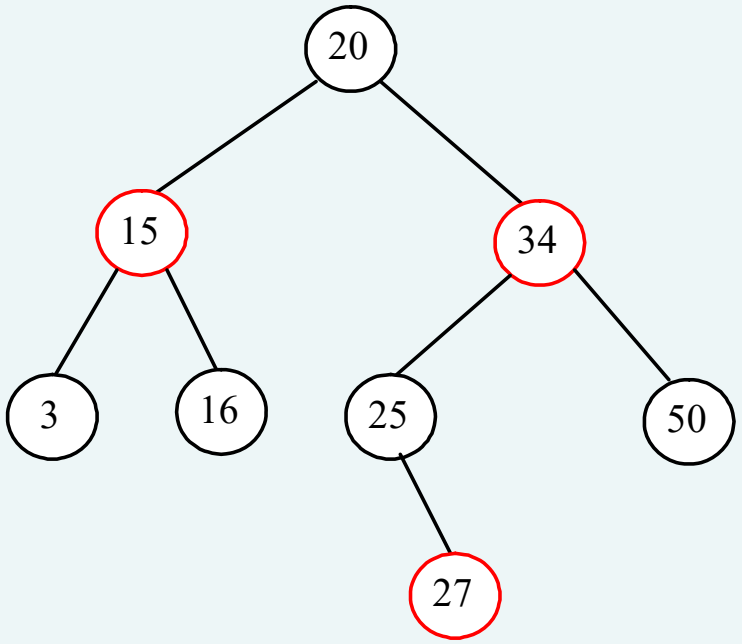
Rotations maintain the inorder ordering of keys:

- $a \in \alpha, b \in \beta, c \in \gamma \Rightarrow a \le A \le b \le B \le c$.

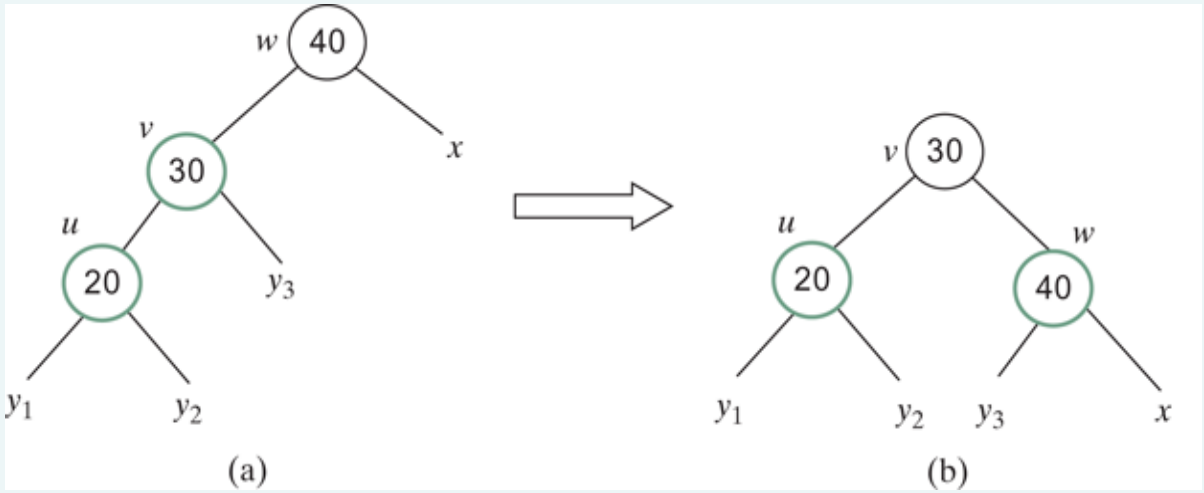
A rotation can be performed in $O(1)$ time.

How to insert an element to red-black tree.

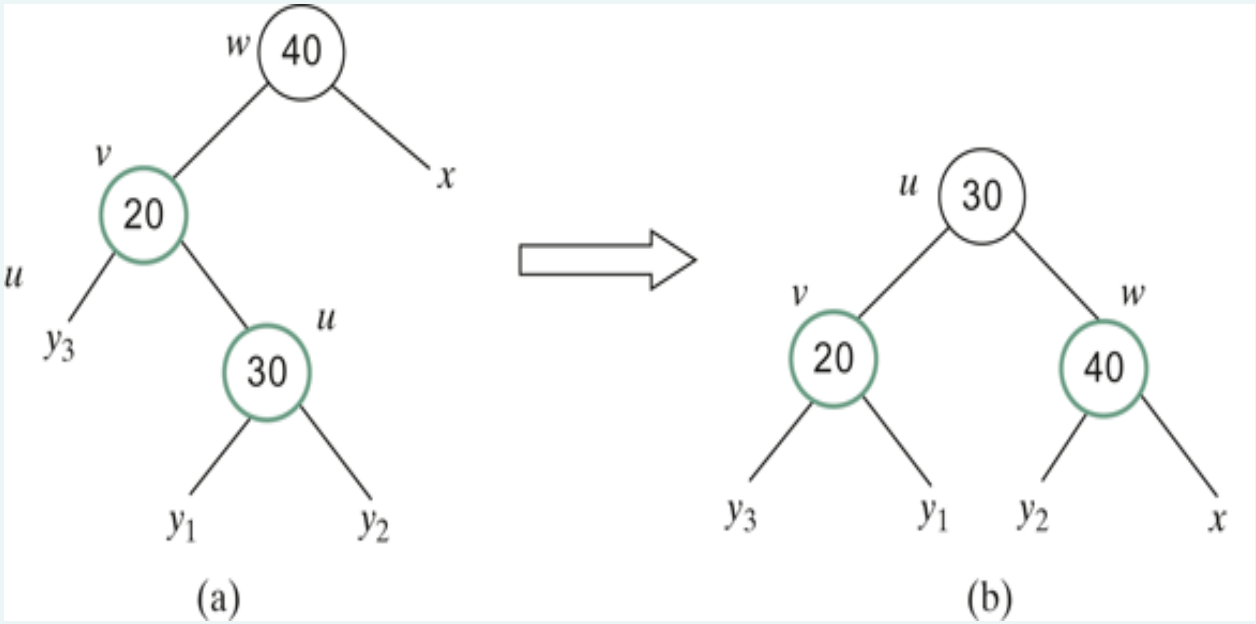
- Let *u* denote the new node inserted, *v* the parent of *u*, *w* the parent of *v*, and *x* the sibling of *v*.
- To fix the double-red violation, consider two cases:
  - Case 1: *x* is black or *x* is null.
    - There are four possible configurations for *u*, *v*, *w*, and *x*



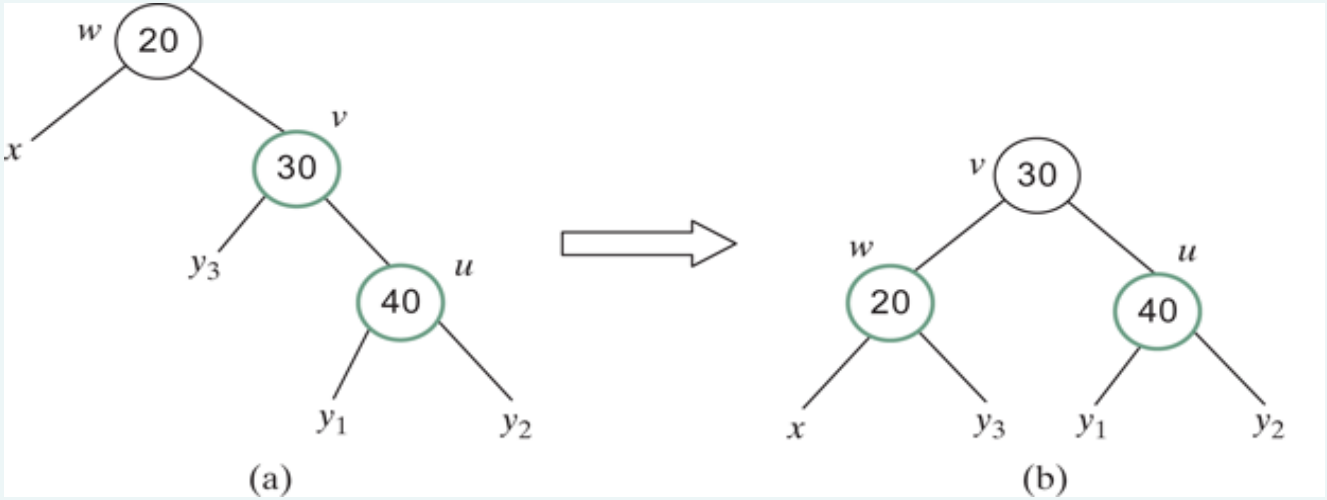
Case 1.1: u<v<w



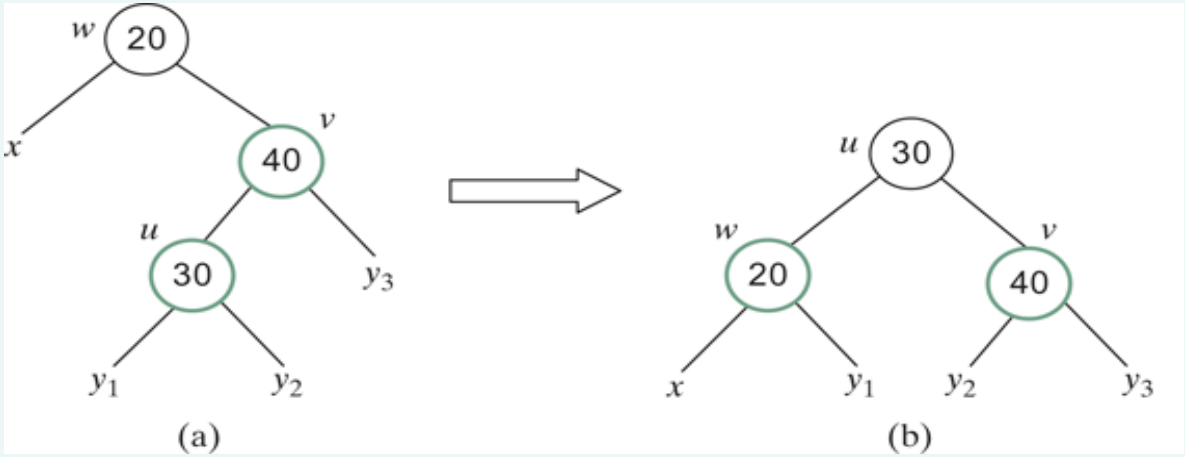
Case 1.2: v<u<w



Case 1.3: w<v<u

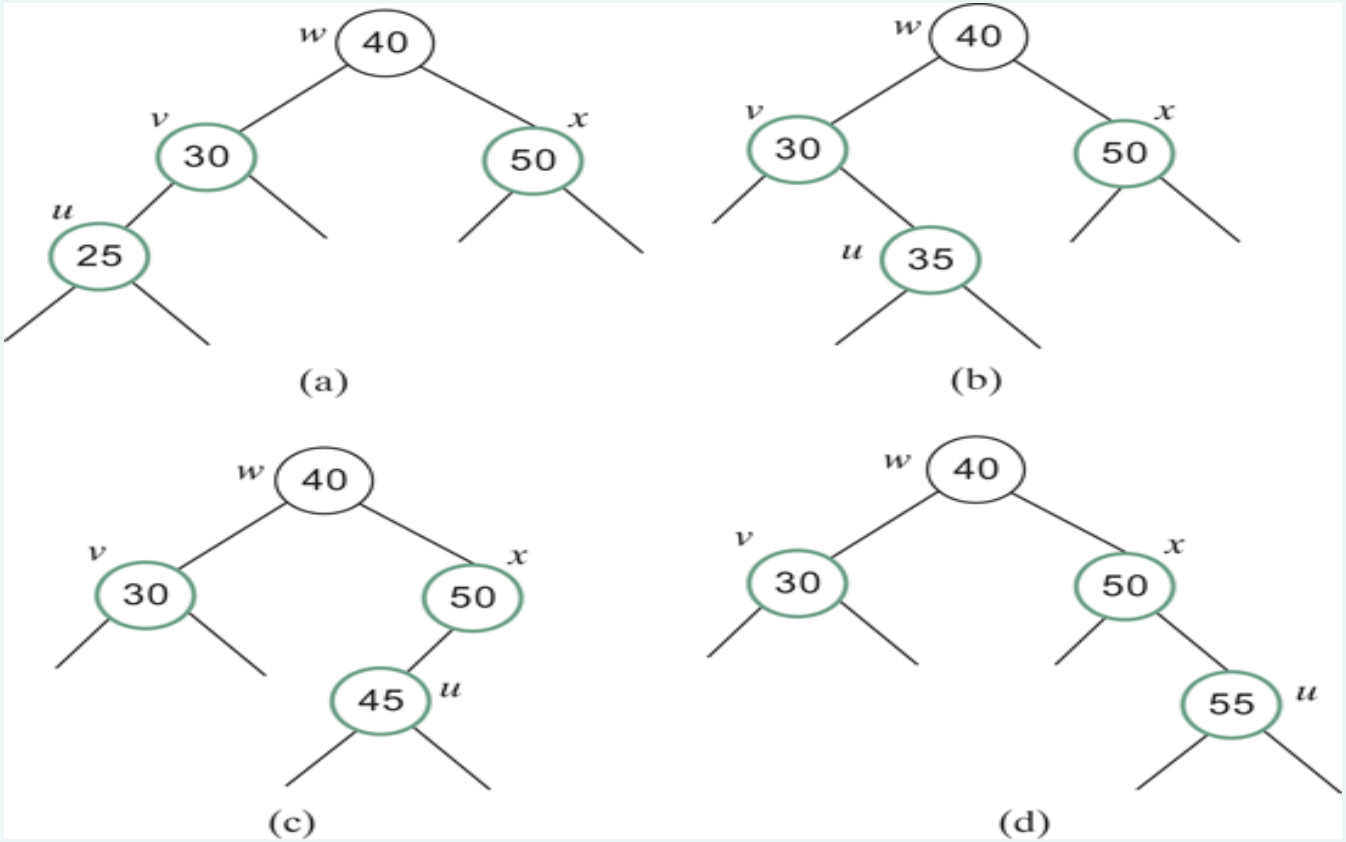


Case 1.4: w<u<v

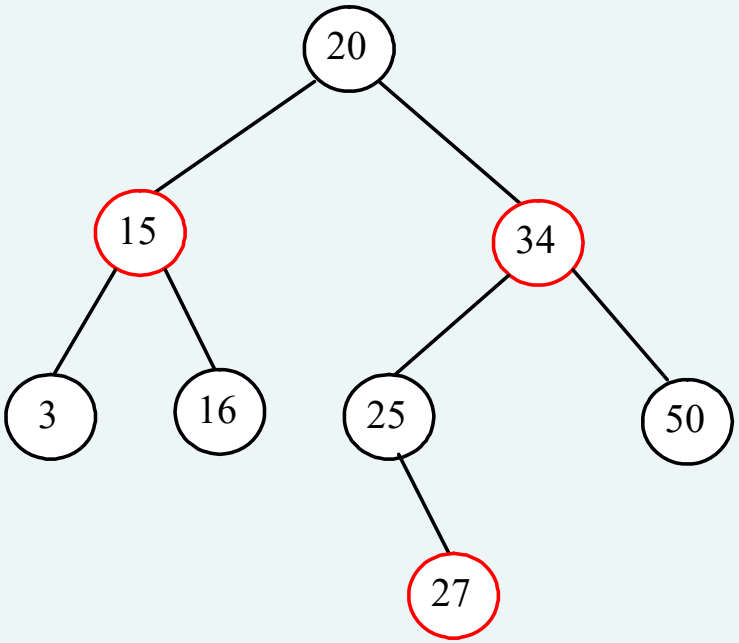


- Case 2: x is red.
  - There are four possible configurations for u, v, w, and x.

Case 2 has four possible configurations.

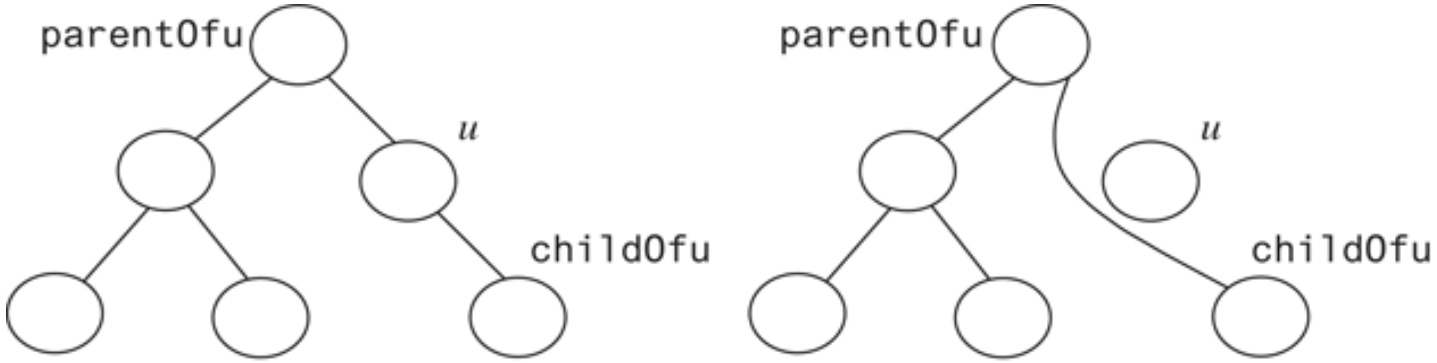


Insert 26



Deleting from red-black tree

1. Search the element in the tree to locate the node that contains the element.
2. Let *u* be the node that contains the element.
   - If *u* is an internal node with both left and right children, find the rightmost node in the left subtree of *u*.
   - Replace the element in *u* with the element in the rightmost node.
   - Now we will only consider deleting external nodes.



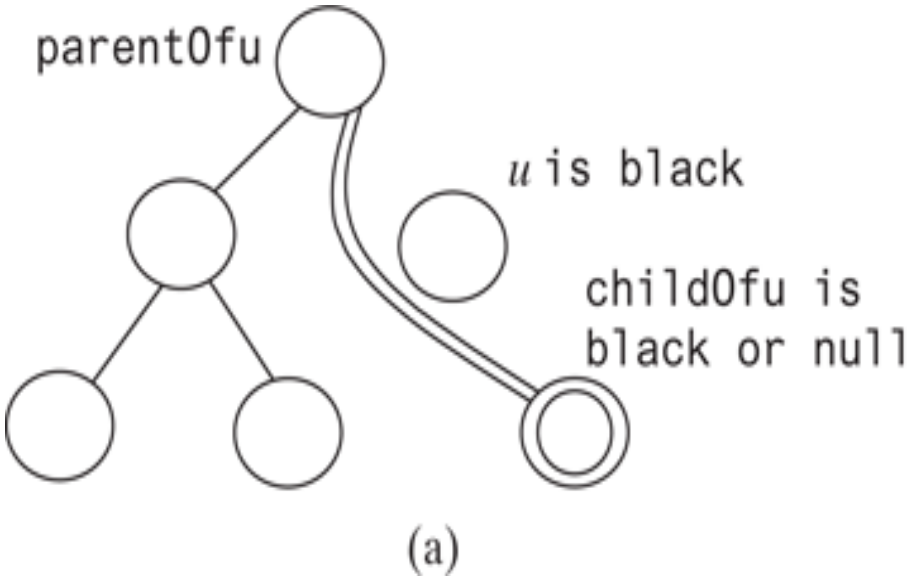
(a) Before deleting *u* (b) After deleting *u*

Let *u* be an external node to be deleted. Since *u* is an external node, it has at most one child, denoted by *childOfu*. *childOfu* may be **null**.

Let *parentOfu* denote the parent of *u*, as shown in **Figure (a)**. Delete *u* by connecting *childOfu* with *parentOfu*, as shown in **Figure (b)**

Consider the following case:

- If *u* is red, we are done.
- If *u* is black and *childOfu* is red, color *childOfu* black to maintain the black height for *childOfu*.
- otherwise, assign *childOfu* a fictitious *double black*, as shown in **Figure (a)**. We call this a *double-black problem*, which indicates that the black depth is short by **1**, caused by deleting a black node **u**.



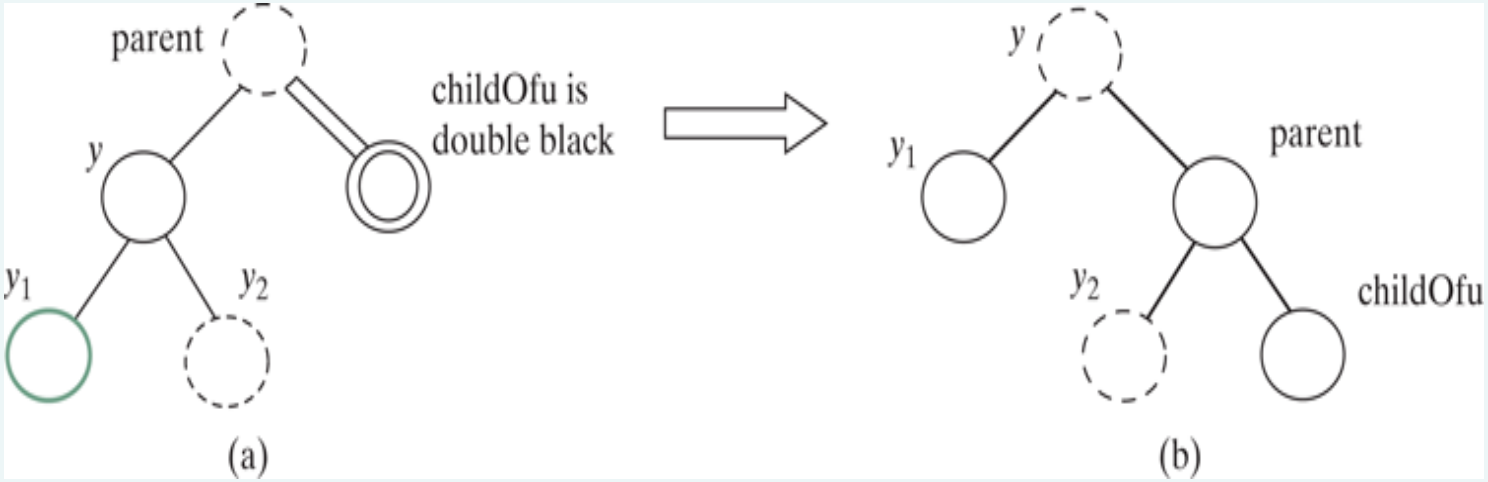
(a)

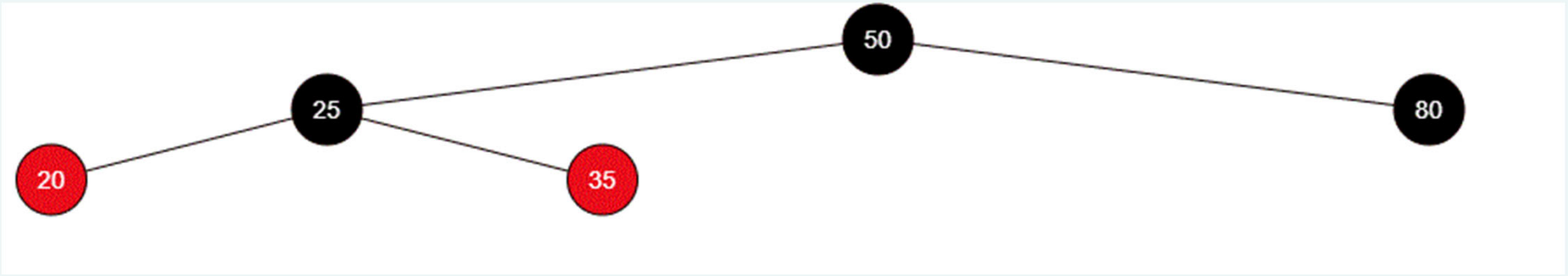
(a) **childOfu** is denoted double black.

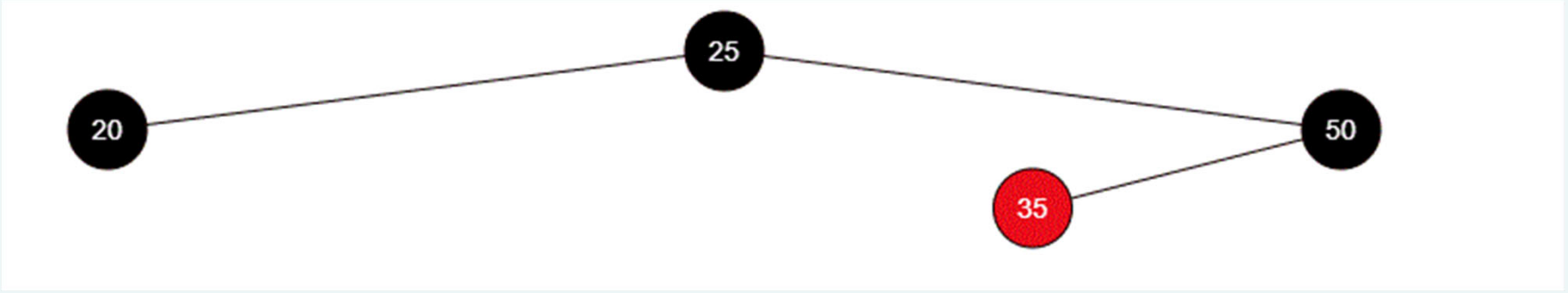
Fixing double-black problem

- Perform equivalent transfer and fusion operation.
- Consider three cases
- Case 1: The sibling y of childOfu is black and has a red child.
  - 4 possible configurations.

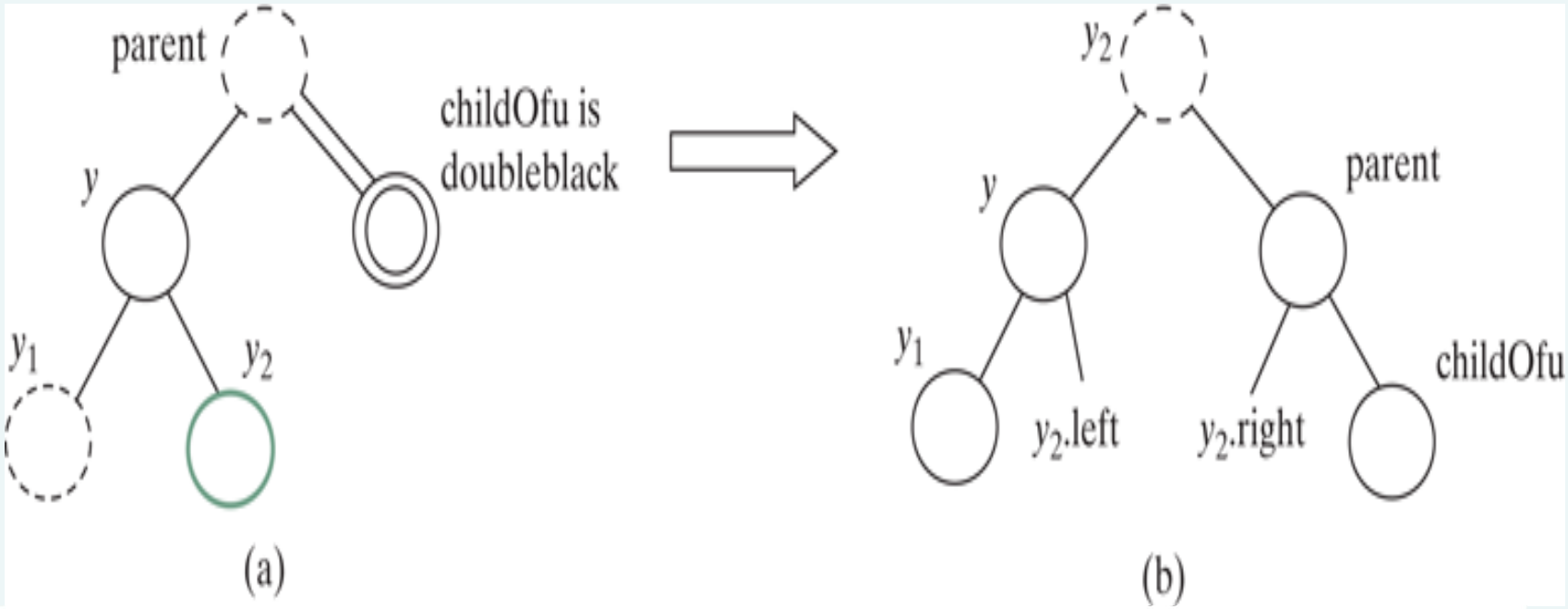
Case 1.1: The sibling y of childOfu is black and y1 is red.

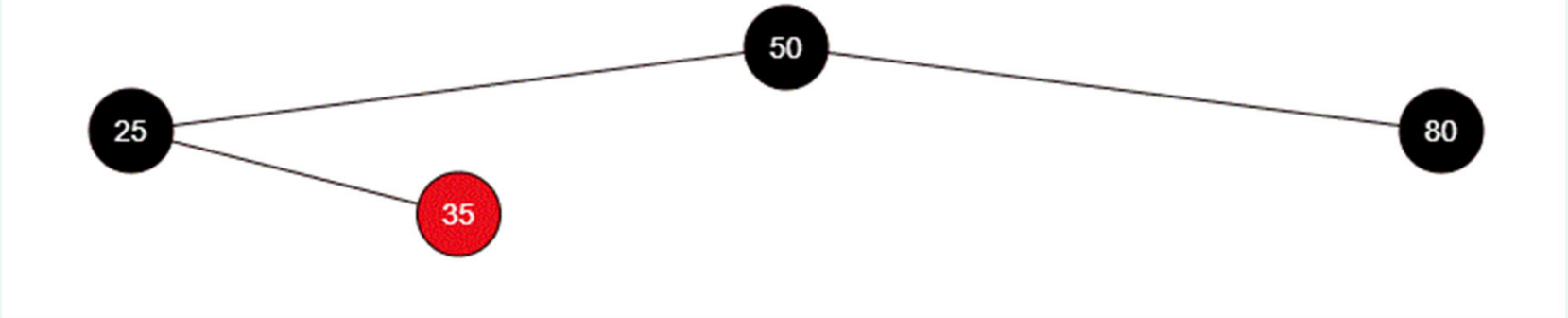


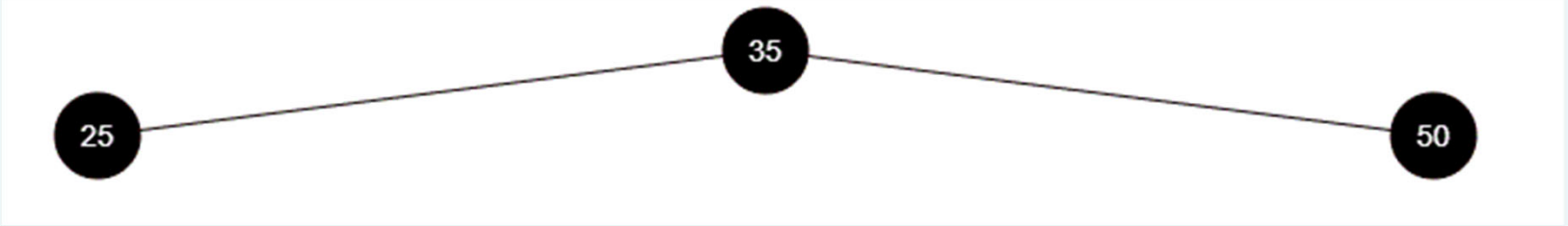




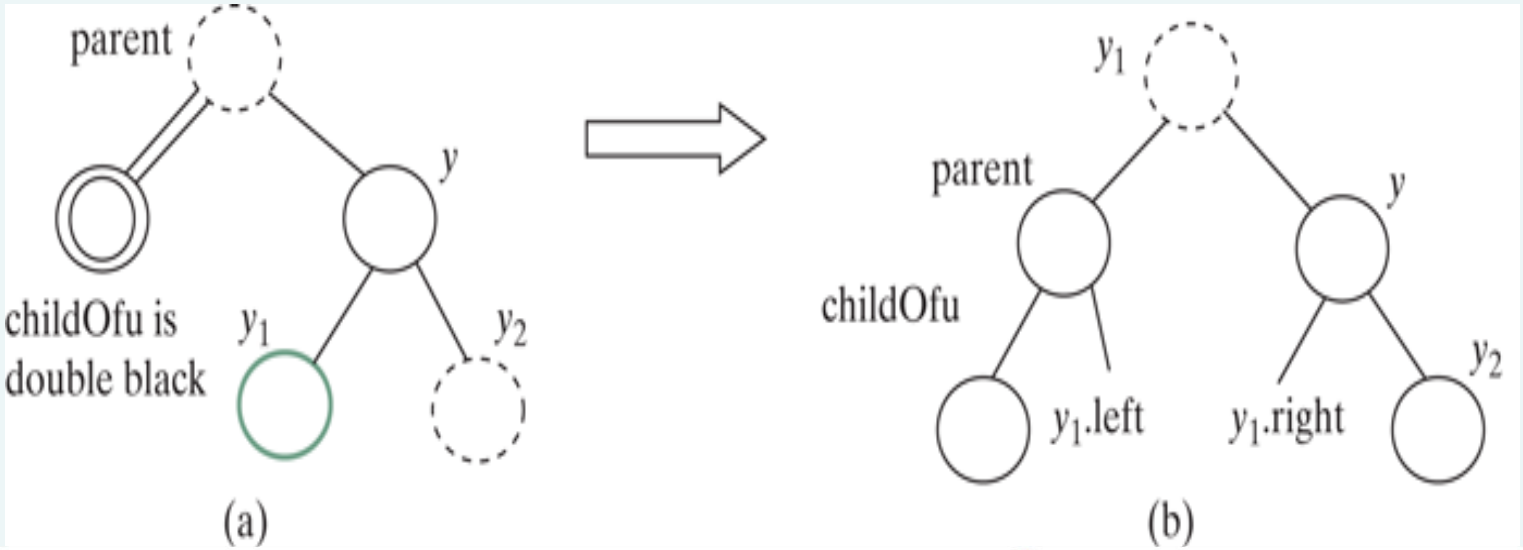
Case 1.2: The sibling **y** of **childOfu** is black and **y2** is red.

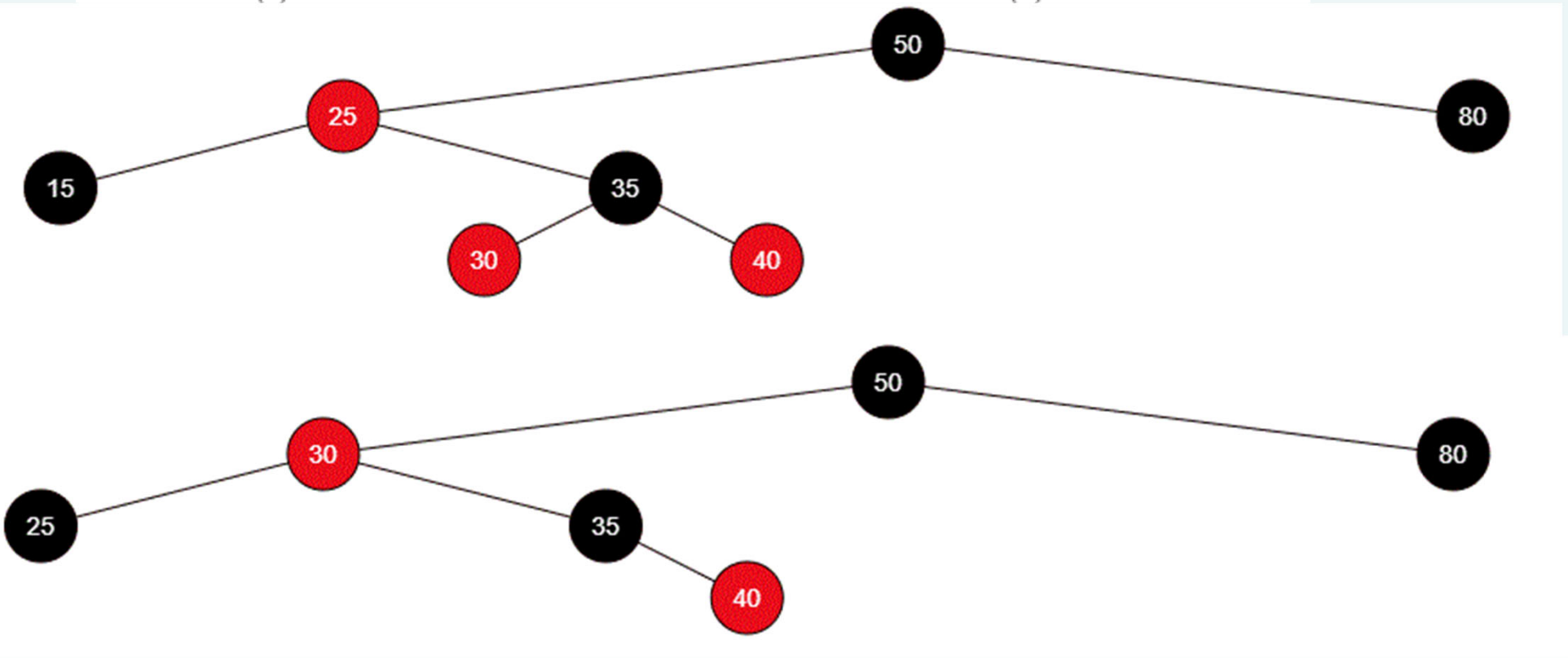




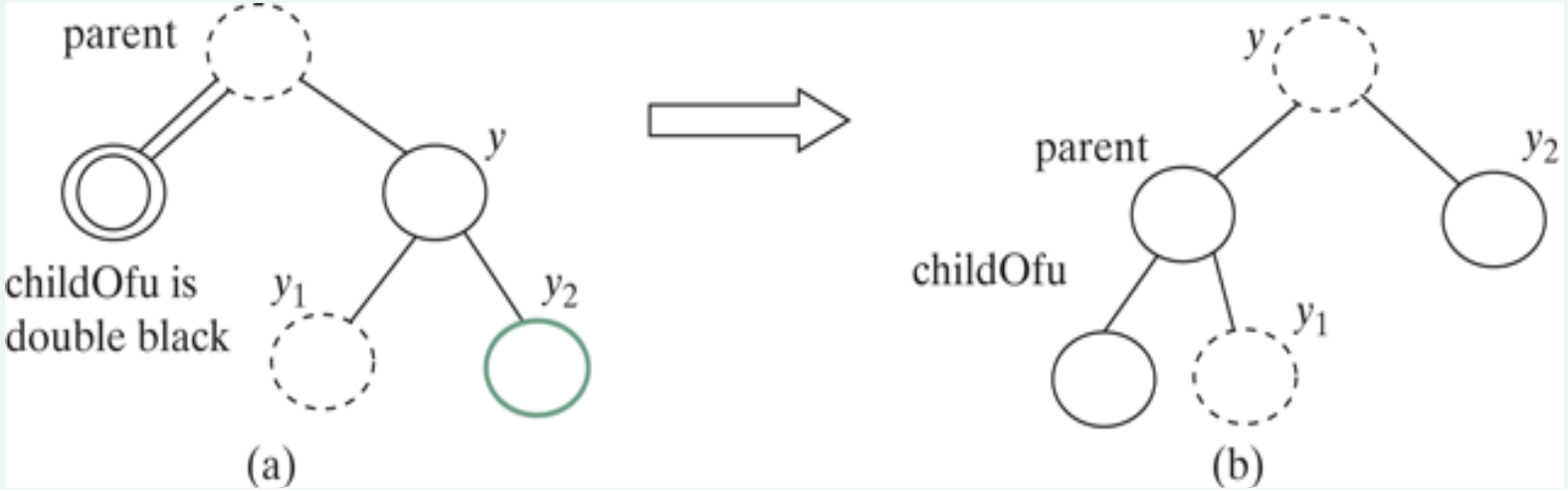


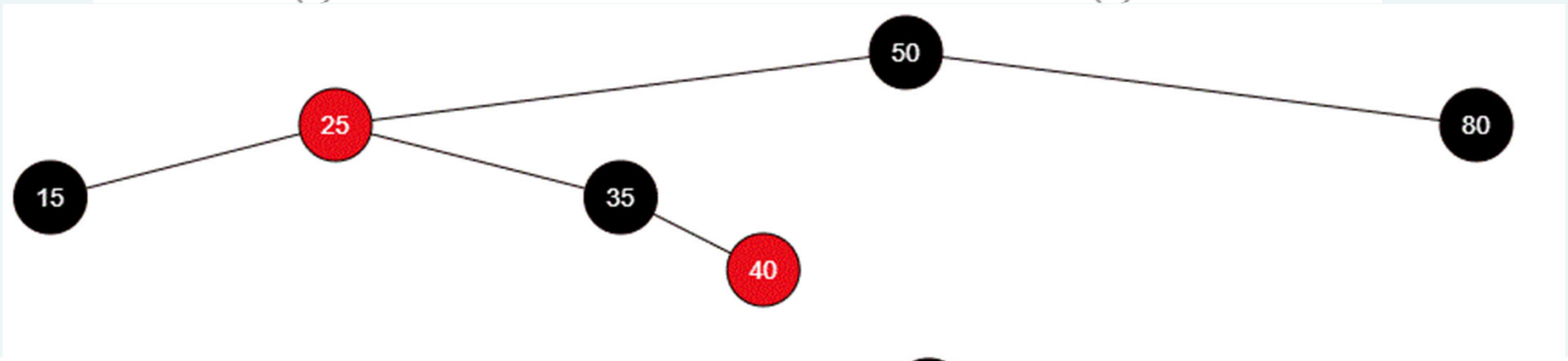
Case 1.3: The sibling **y** of **childOfu** is black and **y1** is red.

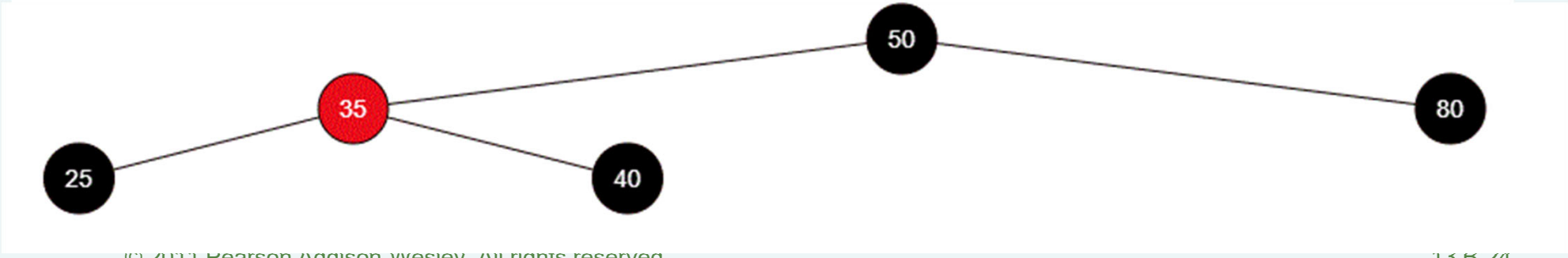




Case 1.4: the sibling **y** of **childOfu** is black and **y2** is red.

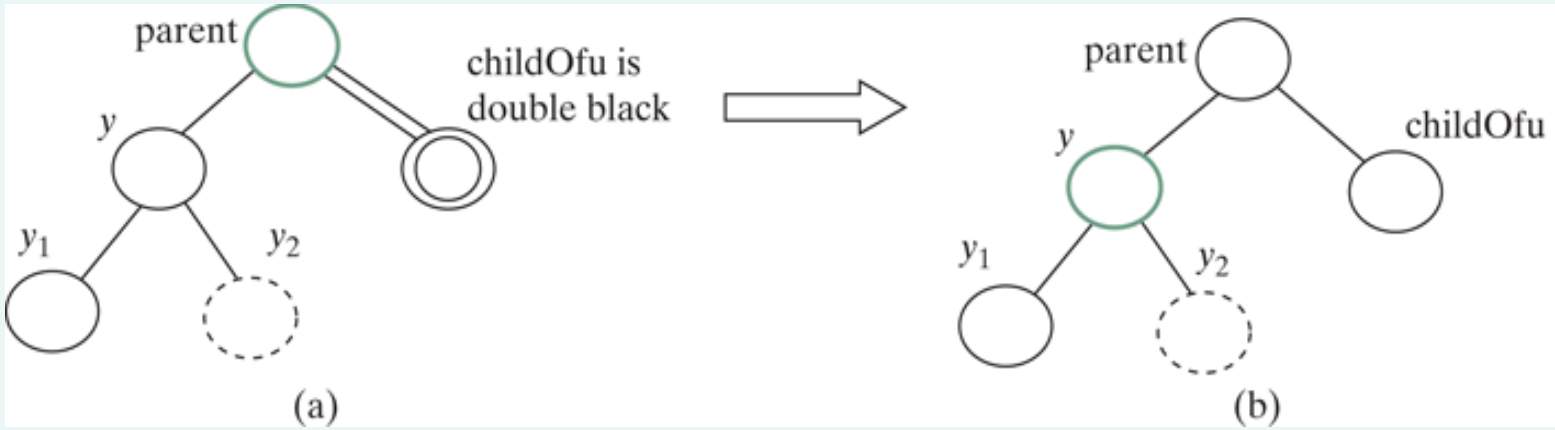


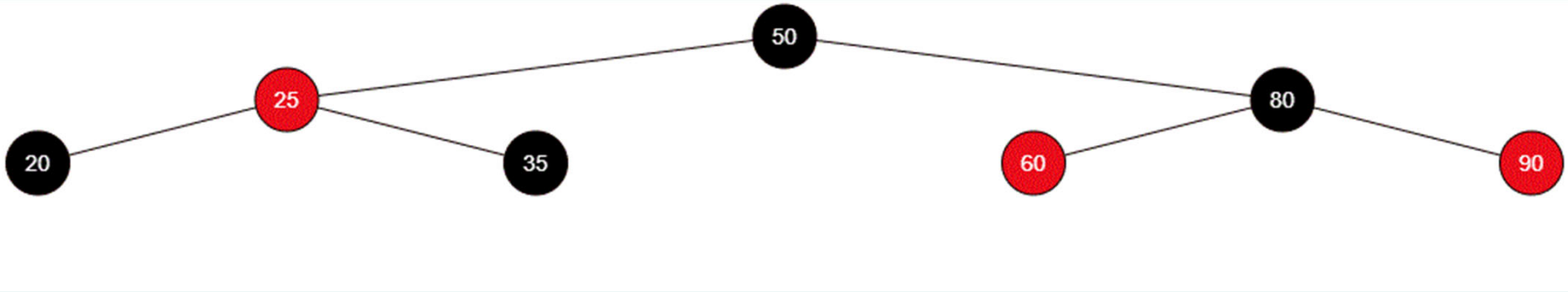


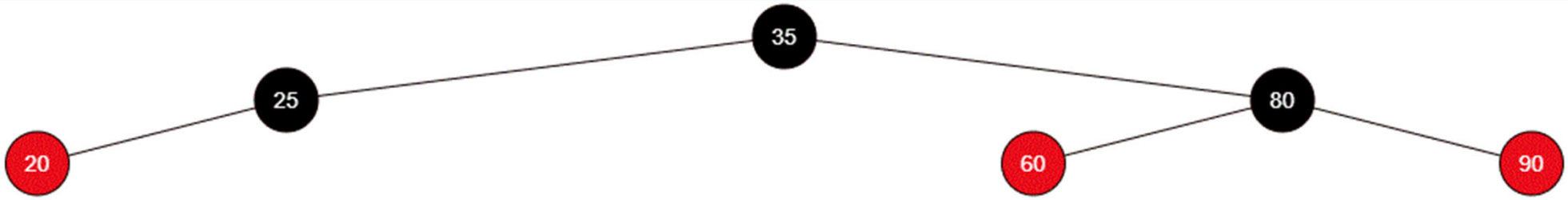


Case 2: The sibling **y** of *childOfu* is black and its children are black or **null**.

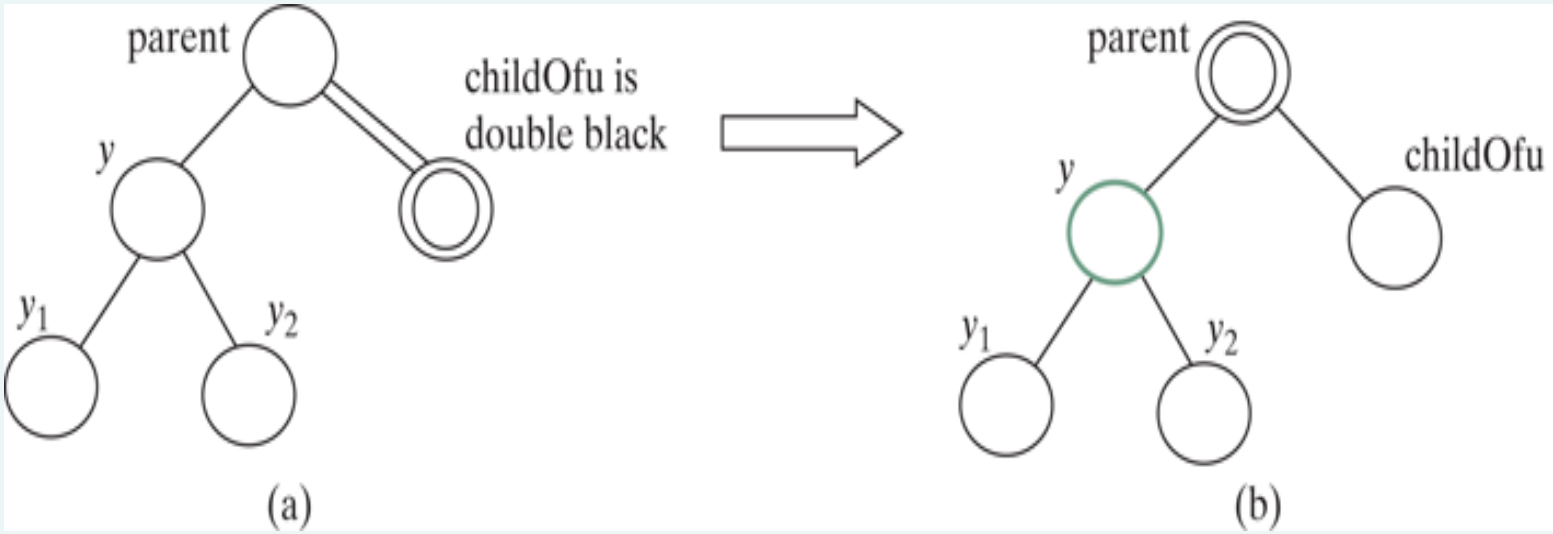
change **y**'s color to red. If **parent** is red, change it to black, and we are done,





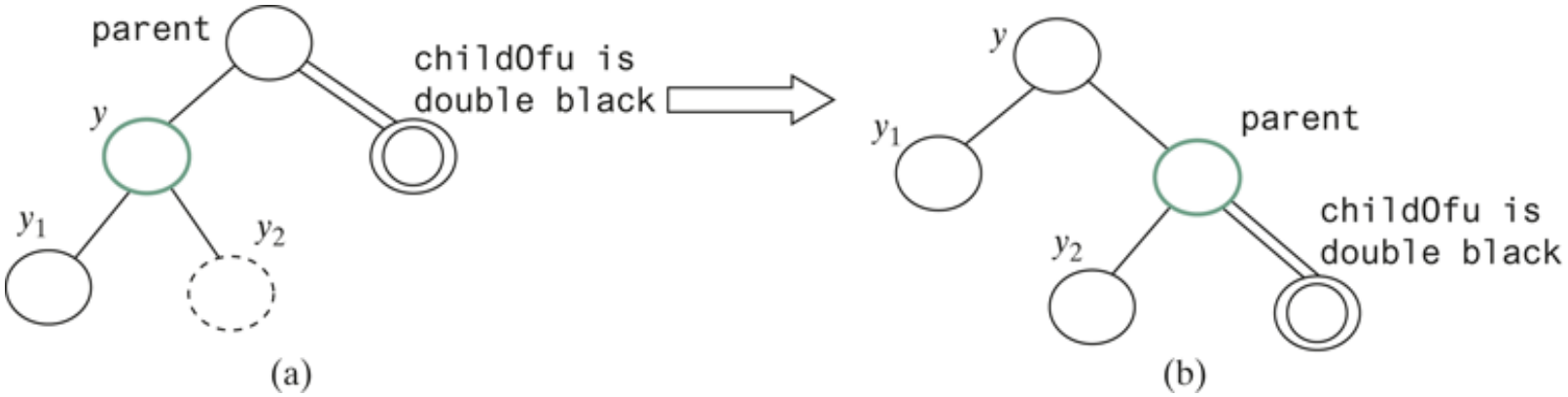


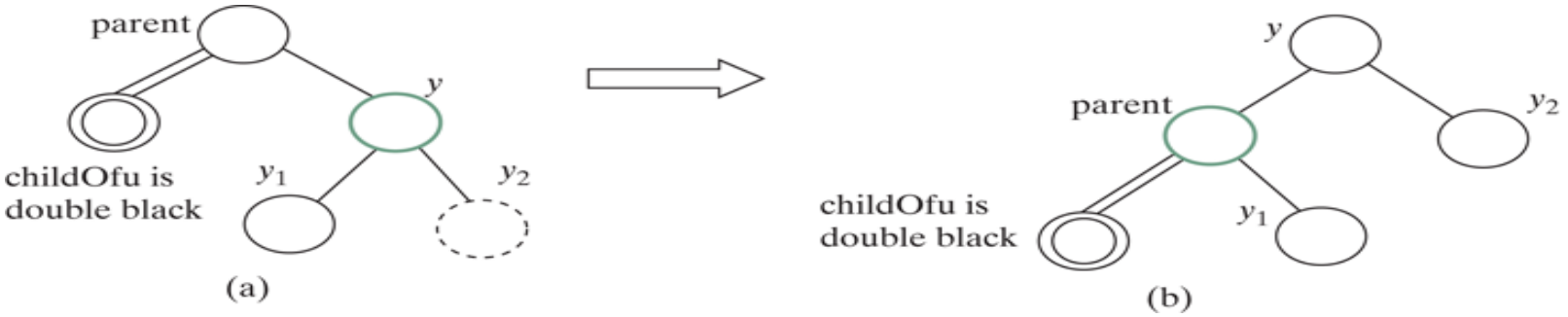
Case 2: Recoloring propagates the double-black problem if **parent** is black.

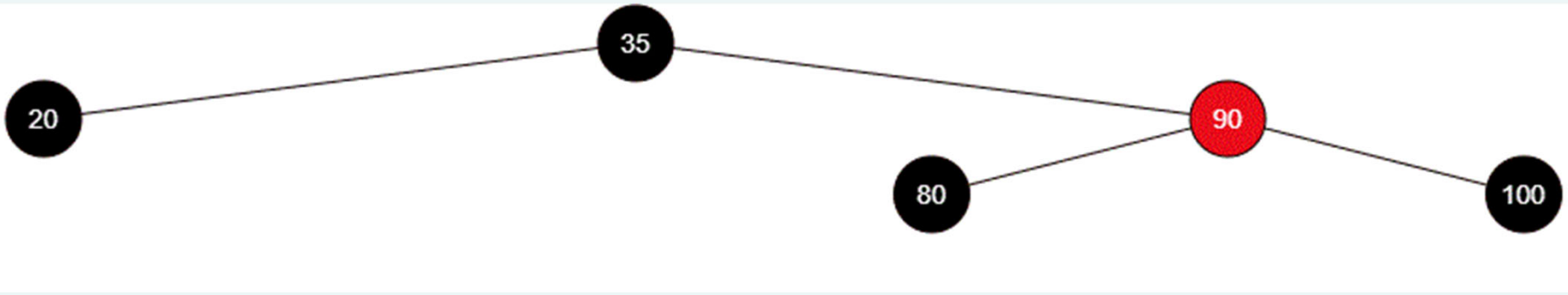


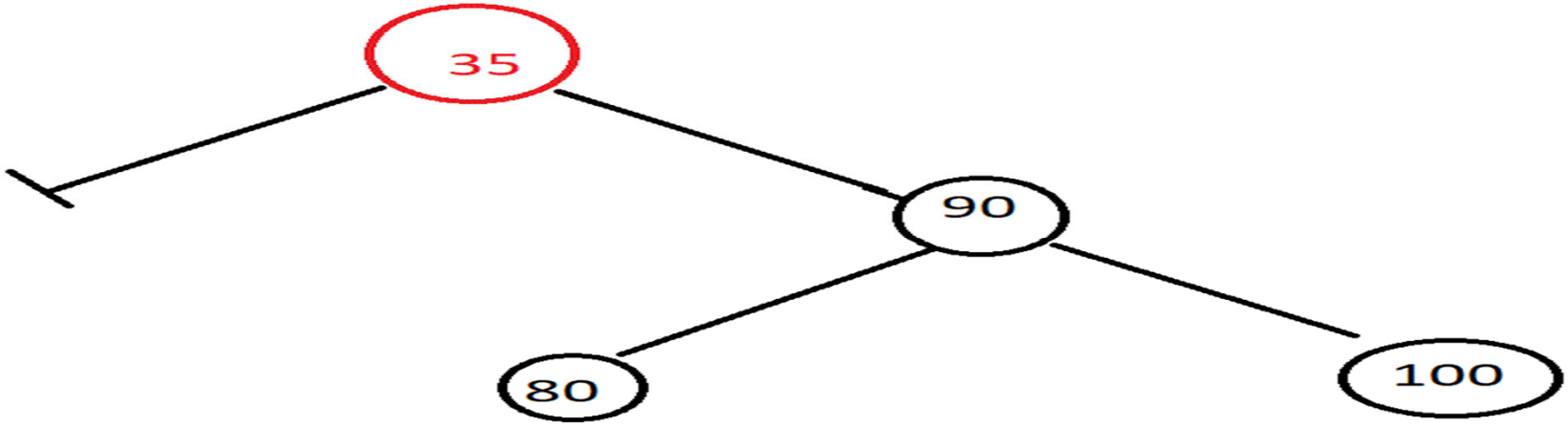
Case 3: The sibling **y** of *childOfu* is red. In this case, perform an *adjustment* operation.

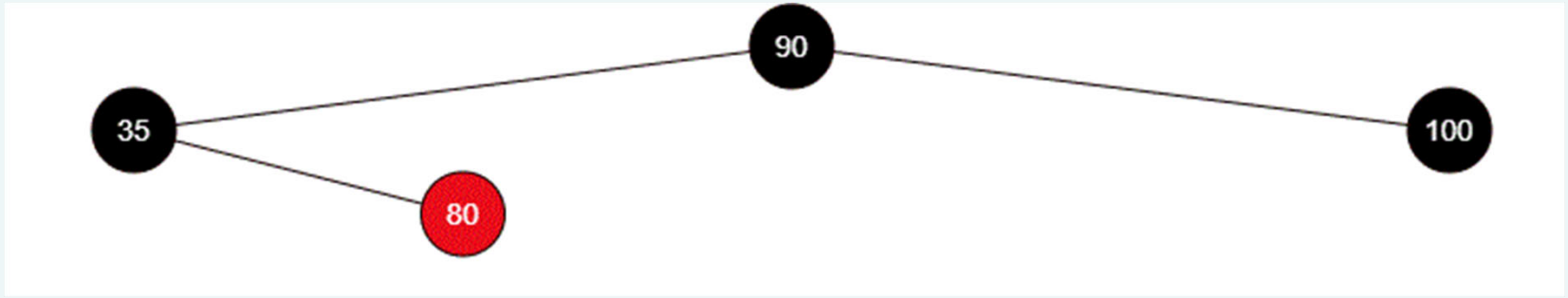
- Case 3.1: If **y** is a left child of **parent**, let **y1** and **y2** be the left and right children of **y.**
- Case 3.2: If **y** is a right children of **parent**, let **y1** and **y2** be the left and right child of **y**,
- In both cases, color **y** black and **parent** red. **childOfu** is still a fictitious double-black node. After the adjustment, the sibling of **childOfu** is now black, and either Case 1 or Case 2 applies. If Case 1 applies, a one-time restructuring and recoloring operation eliminates the double-black problem. If Case 2 applies, the double-black problem cannot reappear, since **parent** is now red. Therefore, one-time application of Case 1 or Case 2 will complete Case 3.



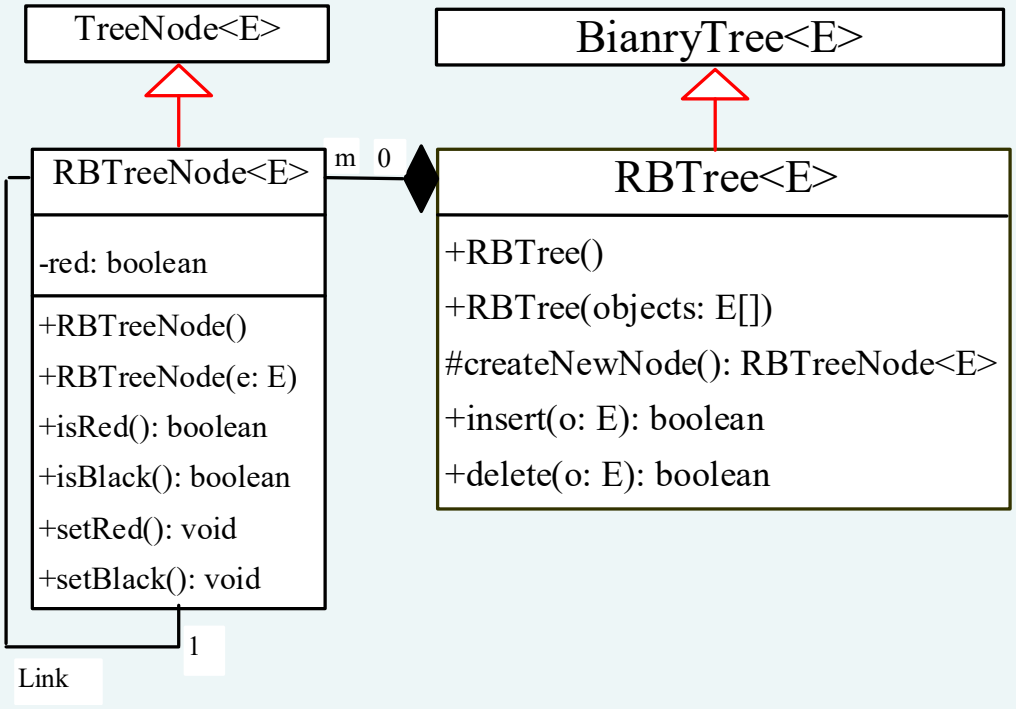








Designing Classes for Red Black Trees



Creates a default red-black tree.

Creates an RBTree from an array of objects.

Override this method to create an RBTreeNode.

Returns true if the element is added successfully.

Returns true if the element is removed from the tree successfully.